

PGR307 Agile project

Part 2 Project Report

Part 3 Reflection Note

Candidate: 9, 15,17, 30,32,35,38

Fall, 2025

Contents

Part 1 Project Result	2
Part 2 Project Report	2
1. Introduction	2
2. Iterations	4
3. Process	17
4. The Way Forward	29
Part 3 Reflection Note.....	31
Group Reflection Note.....	31
Individual Reflection Notes	32
Reference list:.....	33

Part 1 Project Result

Part 1 project result is in a different PDF submission named Result_Final_pt1.pdf.

Part 2 Project Report

1. Introduction

In this project, we were tasked with developing a solution for Peers, with one clear focus: making onboarding for event planners as simple and intuitive as possible.

Peers is a platform that supports event planning and hosting. According to the owner of Peers, *“Peers empower events with innovative judging technology, enhancing audience engagement and creativity for sport, competitions and awards.”* In practice, this means Peers enables events where the audience can interact dynamically, for example, through voting or rating, which strengthens engagement and participation.

Vision

Our vision was to make it effortless for event planners to go from “I want to create an event” to “my event is ready,” with flexible setup options, minimal friction and maximum clarity. The long-term goal is that new event makers can confidently set up events in Peers without feeling confused, overwhelmed, or dependent on external help.

Achievement

We aimed to create a user-friendly and efficient event setup process for event makers in Peers, where users can start from whatever they already have (no information, partial information, or a prepared file) and still complete the setup with minimal friction. More specifically, the solution should provide:

- A clear and intuitive onboarding flow where event makers always understand what to do next.
- Three ways to set up an event:

- AI-assisted chat: the user sets up the event through a chat flow where AI guides the process, suggests content, and helps fill in required information; a checklist is provided.
- Manual setup: the user completes the event setup by filling in the fields directly with the checklist, without AI support.
- Upload-based setup (existing information): the user uploads event information they already have, the system extracts relevant details to pre-fill the setup, and the user then continues and finalises the event through either the manual flow or the AI-assisted flow.
- A dynamic preview alongside setup, so users can continuously see how the event will look while they are configuring it. (*“Dynamic preview” = it updates immediately as you change information.*)
- A checklist for the user, showing what is completed and what is still missing before the event can be finalized. (*“Checklist” = a list of required steps/fields that can be ticked off.*)
- An easy upload and management process for registrations, especially handling uploads for:
 - Athlete registration (participants/competitors)
 - Judge registration (the people who evaluate/rate)
 including clear feedback if required data is missing or formatted incorrectly.

Target audience

The target audience for this solution includes event planners with varying levels of event planning experience, who want to create and manage competition-based events (e.g., sports, contests, awards) where the audience can interact through voting/rating.

- Primary users: Event planners (from beginners to experienced planners) who need a simple onboarding and event setup flow.
- Secondary users: Audience participants who vote/rate during the event, as well as judges and athletes/participants whose registration details are managed through the system.

- Stakeholders: Peers (product owner/company) and any admins/support roles who benefit from reduced onboarding friction and fewer support requests.

Other thoughts

This project matters because onboarding is often where users decide whether a product feels worth using. If the first experience is confusing, users may abandon the platform before ever reaching its core value. Since Peers' strength is interactive judging and audience engagement, making onboarding simple and intuitive is essential for helping event makers successfully launch events and unlock that interaction.

2. Iterations

2.1 Google Design Sprint

To support the project goal from the introduction, making onboarding for event planners in Peers as simple and intuitive as possible, we began with a Google Design Sprint. The sprint helped us quickly understand where onboarding creates friction, explore multiple solution directions, and validate a direction through a prototype and user testing.

Key achievement 1: *We documented the current manual event setup and clarified the core problem*

Early in the sprint, we mapped the existing manual event creation flow. This made it clear where event planners can get stuck, especially when the setup includes many steps and settings that are not self-explanatory. This gave the team a shared understanding of what feels difficult for first-time users and why onboarding needed improvement.

Figure 1: Home page after logging in.

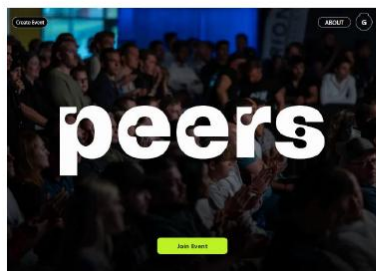


Figure 2: After logging in and clicking “Create Event”

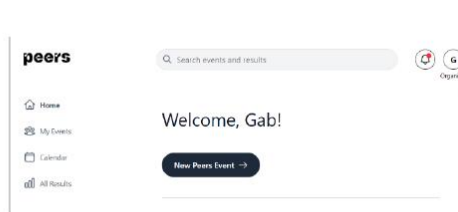


Figure 3: After clicking “New Peers event”

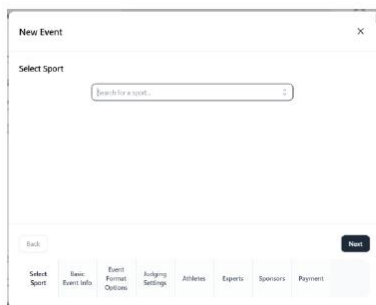


Figure 4: When clicking “Judging settings”

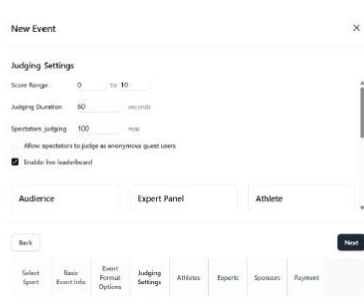


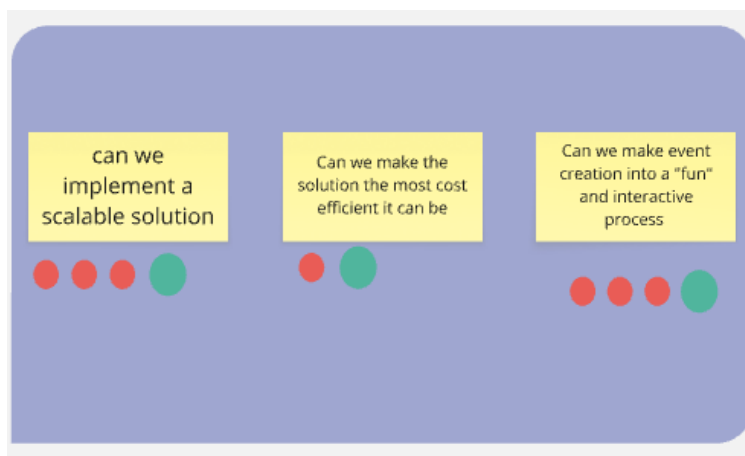
Figure 1-4: Current manual setup flow (shows why onboarding is hard today) Figure 1. Landing Site

Key achievement 2: We used sprint methods to choose a clear focus area

During Day 1, we used structured Design Sprint activities to move from problem understanding to a clear direction:

- Expert input and HMW questions to turn pain points into opportunity questions (HMW means “How might we...”, a way to reframe problems into solvable questions)
- Long-term goal and sprint questions to define what success looks like and what assumptions must be tested
- Journey mapping and focus selection, which highlighted that friction appears early in event creation, including unclear terminology, too many steps, and time-consuming participant setup

This resulted in a focused direction: improving onboarding and event creation so it feels interactive and engaging, while still being intuitive and efficient for event planners.



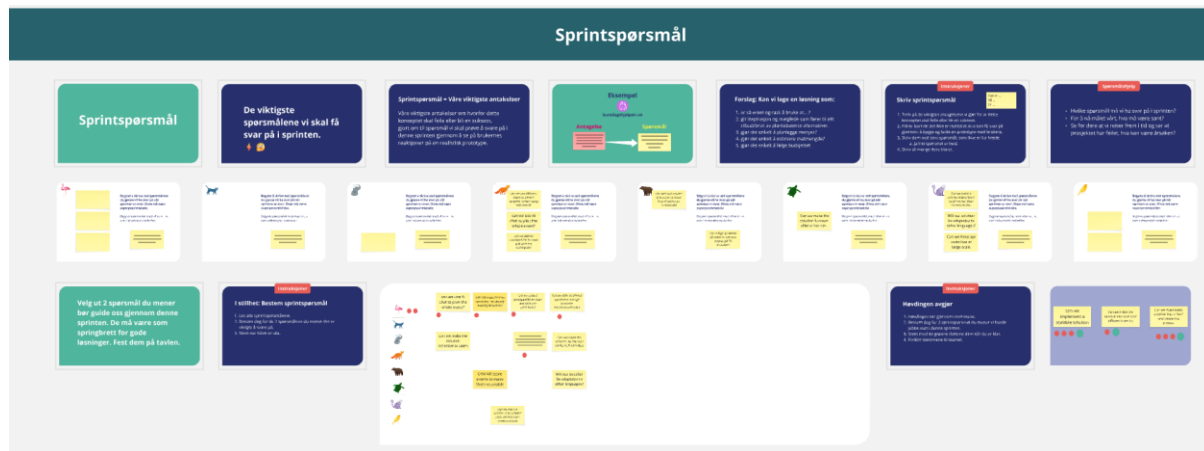


Figure 8: The whole process of the Sprint questions.

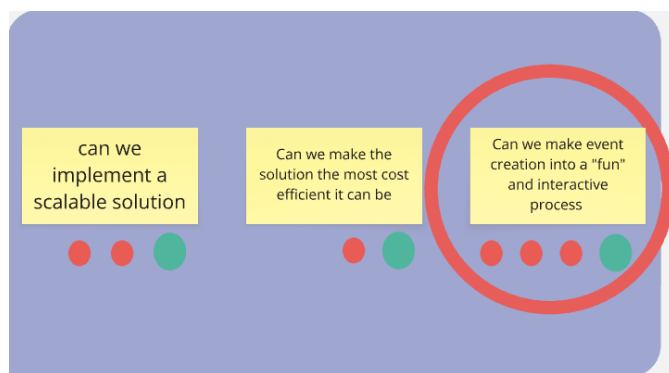


Figure 9: What the chief voted on.

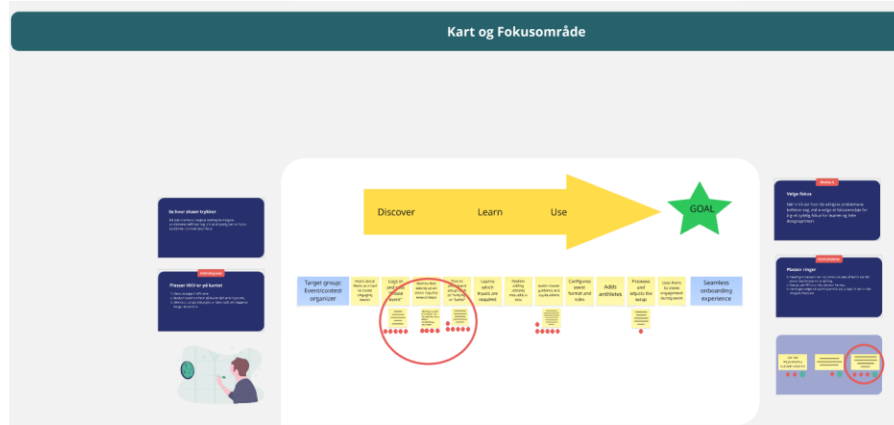


Figure 10: The whole mapping and focus-Area

Key achievement 3: We selected a concept and translated it into a test plan

On Day 2, we moved from ideas to decisions:

- We created a concept gallery and used structured voting to select the most promising concept

- We defined what the prototype must include for testing, then created a storyboard that describes the user journey step by step

This is where the solution direction became concrete and testable.

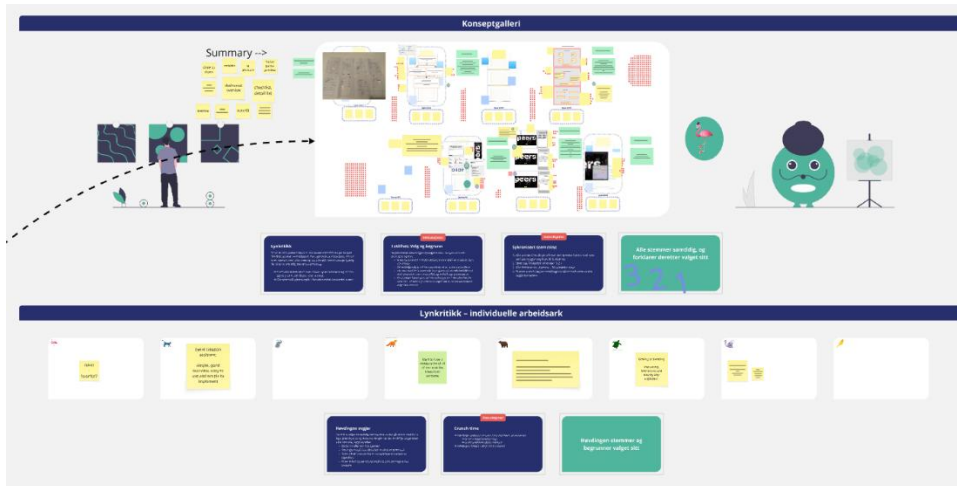


Figure 11: Concept-Gallery

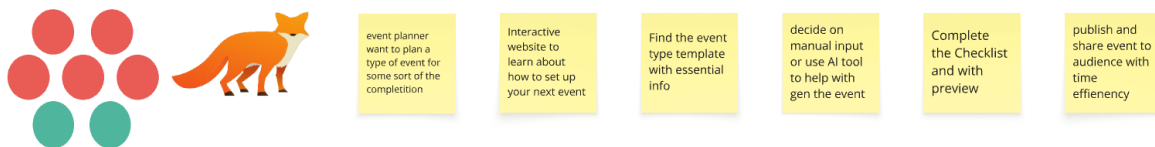


Figure 12: Most voted for User test flow

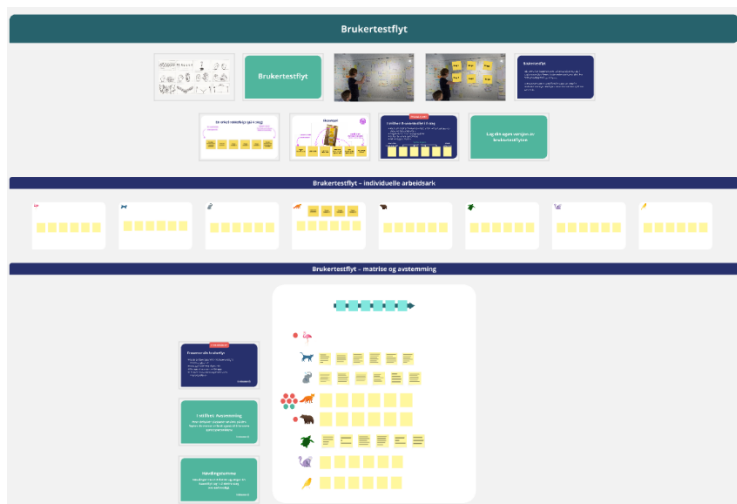


Figure 13: The whole User test flow segment

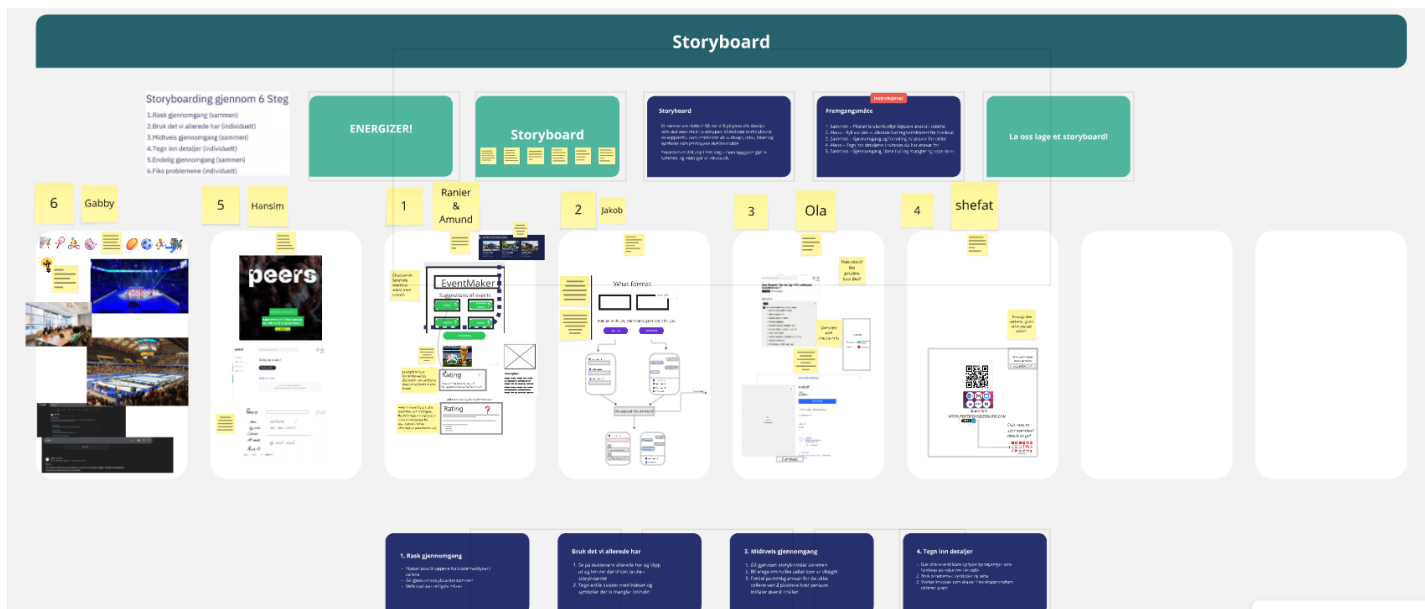


Figure 14: Storyboard

Key achievement 4: *We built a prototype that reflects the project goals*

On Day 3, we built a prototype that demonstrates the full onboarding journey from entering the platform to completing event setup. Work was split across the team, with regular coordination to keep everything aligned.

The prototype included key elements that match the goals described in the Introduction:

- multiple ways to set up an event (AI-assisted chat, manual setup, and upload-based setup)
- a dynamic preview alongside setup, so users can see how the event looks while editing
- a checklist that shows what is complete and what is missing
- support for managing uploads for athlete and judge registration data

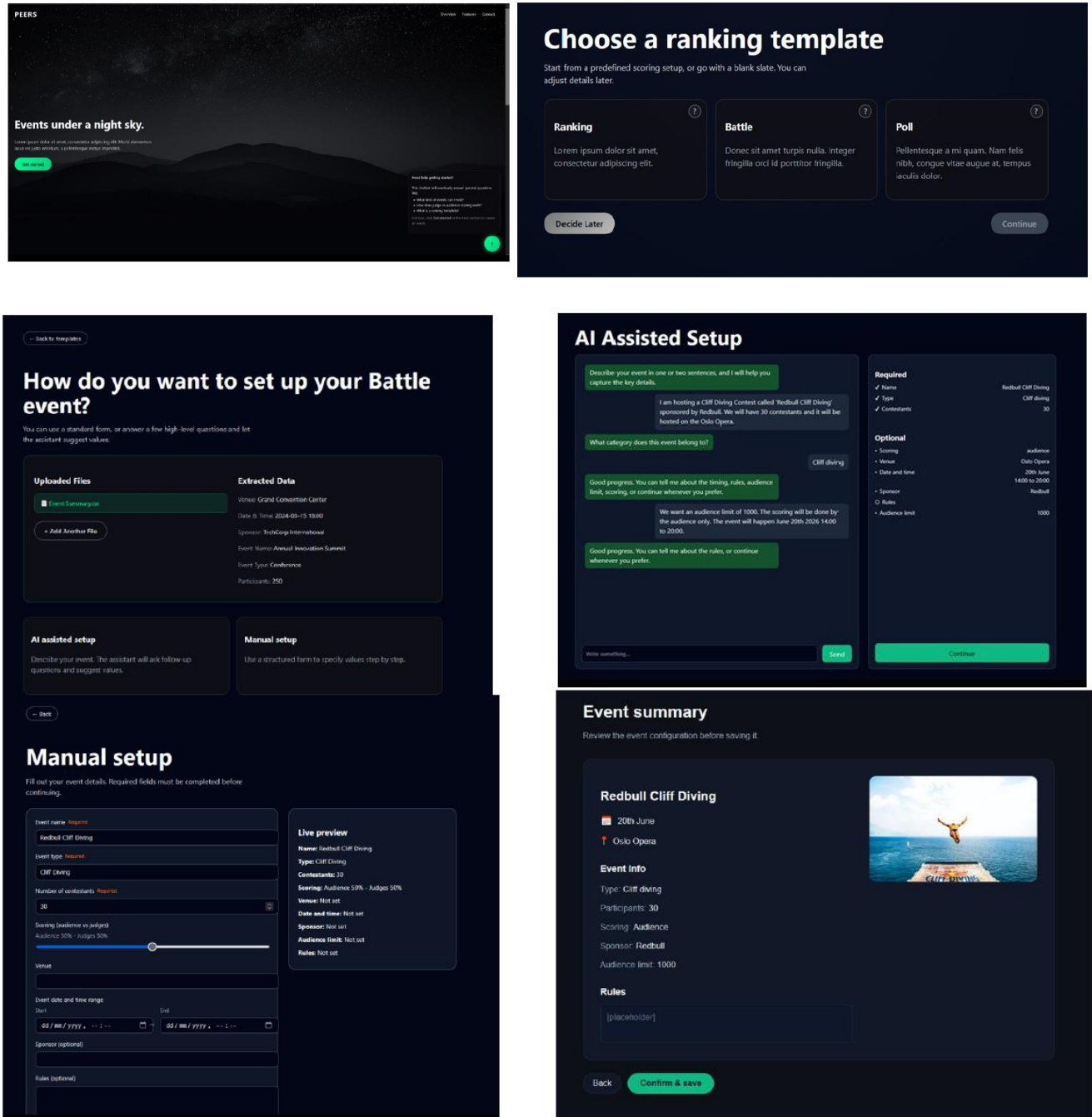


Figure 16: Prototype user journey

Key achievements 5: We tested the concept and identified what needed improvement next

We user-tested the prototype and collected feedback on clarity, usefulness, and whether the onboarding felt easier than a traditional setup. The concept was generally received positively, and users highlighted the value of having structure and visibility during setup, especially

through a checklist and live overview. At the same time, testing revealed clear improvement areas, such as unclear terminology, parts of the import/upload flow, and the need for a clearer explanation of what Peers is on the landing page.

User 1	User 2	User 3	User 4	User 5	Avg	Question
4	4	3	4	3	3.6	1. What was your first impression when you opened the landing page?
2	4	4	4	3	3.4	2. Did you understand what the service does?
2	2	5	3	4	3.2	3. How did you experience the interaction with the AI assistant?
4	4	5	4	4	4.2	4. Did the chatbot feel helpful and relevant for the task?
4	3	4	4	3	3.6	5. Did it feel easier than how you would normally set up an event?
4	5	5	3	5	4.4	6. Did the AI feel tailored to your needs?
5	4	4	2	3	3.6	7. Was anything confusing or unclear during the process?
4	4	5	2	4	3.8	8. Did you trust the service/Peers Event based on the test?
4	4	4	4	4	4.0	9. Would you personally want to use this service in the future?
1	3	3	1	3	2.2	10. Would you consider paying for it / subscribing?

Sprint 3	User 6	User 7	User 8	User 9	User 10	Avg
	4	4	4	4	4	4.0
	4	3	2	4	3	3.2
	5	5	5	5	4	4.8
	5	4	4	5	5	4.6
	5	4	3	4	4	4.0
	4	5	5	5	5	4.8
	4	3	4	5	4	4.0
	4	3	4	4	4	3.8
	4	5	3	4	3	3.8
	4	2	3	4	3	3.2

Figure 17: User test results/average scores, 1 is the lowest, and 5 is the best.

User 2	1	Likes the design, thinks it looks professional, and the colors look good.
User 2	2	Gives "night event" vibes → makes him think more about parties, like night events.
User 2	3	Didn't understand the difference between Battle, Poll, etc. Also didn't know there was a "Decide later" button.
User 2	4	Liked that there was an AI; manual setup was too boring and took more time.
User 2	5	The AI was helpful but complained that it didn't understand anything and was a bit difficult to work with. However, liked that there was
User 2	6	Manual setup was okay, but he would rather use the AI, and it would have been much easier if the AI worked properly.
User 2	7	It wasn't fully clear, but the overall experience was good.
User 2	8	Trusted the process, but not the AI because it didn't work that well.
User 2	9	Would use it himself.
User 2	10	Would probably not pay for a subscription because people don't often create events, but maybe a maximum of 100–150 NOK
User 3	1	Empty, big room, doesn't give you any impression of what it's about. – 2 on the prototype, 3 on the original page.
User 3	2	From looking at the page you don't really pick up what they do. With the original page, it was a bit more understanding because of the background
User 3	3	Easy, you understand what happens next.
User 3	4	Yes, because it does what it's supposed to do for me.
User 3	5	Yes, it feels time-efficient. I like the checklist feature.
User 3	6	Yes, it gives me options, feels time-efficient, and gets the job done.
User 3	7	Confusion around the Import File part — shouldn't that come after choosing whether you want assistance or manual setup?
User 3	8	Gives a professional impression.
User 3	9	Yes, because it seems time-efficient. Automated, which can save me time and gives me a clear overview.
User 3	10	Not for small events, but for larger events the willingness is higher.

Figure 18: User comments / qualitative feedback summary

2.2 More detailed description (Appendices)

- To keep the main report concise while still documenting the work in full detail, we include the project's weekly documentation as appendices. These documents contain

the detailed descriptions of activities, decisions, sprint outputs, and reflections throughout the project.

- **Appendix A:** Documentation 1 (Google Design Sprint / Week 1). Includes sprint activities, mapping, concept sketches, storyboard, prototype development, and user test results.
- **Appendix B:** Documentation 2 (Sprint 1 / Week 2).
Includes sprint planning, backlog work, Definition of Done and acceptance criteria, progress updates, sprint review, and sprint retrospective.
- **Appendix C:** Documentation 3 (Sprint 2 / Week 3).
Includes sprint goal, development progress, integration work, sprint review results, and sprint retrospective.
- **Appendix D:** Documentation 4 (Sprint 3 / Week 4).

In the following section (2.3), we provide a concise sprint summary based on these appendices, focusing on goals, key outcomes, and what changed between iterations.

2.3 Scrum sprints (Week 2–3) — Concise summary

After the Google Design Sprint, we continued the project using Scrum with weekly sprints, using sprint planning, daily standups, a sprint review, and a sprint retrospective as the main rhythm.

In both sprints we documented progress through a task board, product backlog, burndown charts, and time logs (see Appendix B, C and D).

Sprint 1 (Week 2): Establish the foundation

Purpose: Build the basic framework so we could add and integrate features in the next sprint.

- **How we organized:** We appointed a Scrum Master, agreed on group ground rules, and decided to meet daily for standups.
- **Team structure:** Because we were seven members, we split into two subgroups: frontend and backend, to reduce overlap and clarify responsibilities.
- **Quality alignment:** We defined a Definition of Done and Acceptance Criteria early on (for example: flow works, user can complete steps, tested, reviewed by teammates).

Acceptance Criteria will be updated continuously when new backlog items got introduced in future sprints.

- **Planning and estimation:** We planned in Scrumwise and estimated using Planning Poker and story points.
- **Sprint goals:** Create a database, connect the database with the AI agent, refactor frontend code, and improve key frontend pages (such as the post-registration dashboard).
- **Sprint review (user test):** Because the system was not fully integrated yet, the tester had to test the frontend flow and the AI agent separately. Feedback focused on clarity issues in the landing page and uncertainty about what to do on certain screens. The AI agent was described as somewhat error-prone, but still helpful, and the tester preferred it compared to manual setup. One specific UI point raised was the need to improve scrolling behaviour in the chat area.
- **Sprint retrospective:** We noted we were too cautious when selecting backlog items, and some team members felt less involved than they wanted. At the same time, communication and early Git/workflow rules were highlighted as strong positives, and task board usage improved during the sprint.
- **Outcome:** We achieved the main goal of establishing the basic framework for both backend and frontend, setting us up for integration work in Sprint 2.
- **Minutes (Sprint 1):** Planning 5 hours, review 1 hour, retrospective 45 minutes.

Sprint 2 (Week 3): Integration and stability

Purpose: Merge frontend and backend into a unified workflow and establish a stable end-to-end system.

Sprint goal: Integrate remaining frontend components with the backend, connect the AI agent to the frontend, refine UI, complete essential backend logic, and reduce technical debt.

Execution highlights: We reviewed backlog items together in Scrumwise to align on requirements, then distributed tasks across the team.

Work during the week included refactoring database setup, adding new fields and judging types, working on participant list handling (CSV/Excel support), improving the dashboard/navigation and landing page, and pushing the frontend-backend merge forward. We also hit practical integration challenges where the merged solution did not work consistently across all team members' computers, which temporarily slowed progress. A key milestone was successfully connecting the AI agent to the frontend.

Sprint review (user test): We used the same tester as Sprint 1, and this time the frontend and backend were fully integrated. The user reported the experience felt much smoother, and the improved interaction with the AI agent stood out positively.

The user also identified smaller issues to refine, including UI overlap (FAQ button) and missing "scroll stopping" in the checklist during AI conversations.

Sprint retrospective: Communication, task management, and morale remained strong, but absences due to sickness affected cross-team work and caused two backlog items to remain unfinished and be carried over.

Minutes (Sprint 2): Planning 2 hours, review 30 minutes, retrospective 30 minutes.

Sprint 3 (Week 4) :Final polish and delivery readiness

Purpose: Finalize the solution by fixing remaining bugs and polishing minor functions, while also completing the written report and oral presentation.

How we organized: We followed the same approach as earlier sprints by selecting backlog items that were realistic within the shorter timeframe, estimating point values, and aligning work through daily standups. Some team members focused on report writing, while others focused on completing backlog items and improving the solution.

Planning and estimation: We created and selected backlog items we believed could be completed within the sprint, then estimated point values before starting execution.

Sprint goals: Implement "nice-to-have" features, polish the existing solution, and focus on identifying and fixing remaining bugs. In parallel, outline and complete the report sections and prepare the oral presentation.

Execution highlights: The sprint review with Peers happened earlier than expected (Day 2 instead of Day 3), but we still delivered a successful demo. After the review, we discussed feedback in the standup and used it to guide the remaining sprint work. The last day focused on finishing remaining tasks, delivery testing, and finalizing report sections.

Sprint review

- We conducted a Sprint Review with the owner of Peers as a live demo with four team members present, lasting about 45 minutes.
- The owner responded positively to the three setup options, especially the upload-based flow where event planners reuse existing information, the system extracts relevant details, and the user continues with manual setup or AI-assisted chat.
- During the demo, the AI agent temporarily stopped responding due to an error, but we resolved it quickly and continued. The owner asked several questions to both frontend and backend teams.
- Despite minor issues, the owner said this iteration felt significantly improved overall, and the improved AI interaction made a strong impression.
- Comparing with another group's more polished UI highlighted that we should have allocated more time to UI refinement and final product polish.

Sprint retrospective

- **What went well:** Team motivation stayed high, the group completed the planned backlog items for the week, and attendance/contributions were consistent throughout the sprint.
- **What didn't go well:** Unexpected bugs required more time than planned, often tied to earlier design decisions. In addition, report work became harder to coordinate because many sections had to be written and the group was large.

Outcome: A successful final demo with the product owner, confirming that the sprint focus areas improved the user experience and moved the solution in the right direction.

Minutes (Sprint 3): Sprint planning 2 hours, sprint review 45 minutes, sprint retrospective 30 minutes.

3. Process

How have you used Scrum?

Our group followed the Scrum framework throughout the project (Ruben, 2012), as illustrated in Figure 19. The figure shows the different artifacts and ceremonies we would go through the Scrum Process.

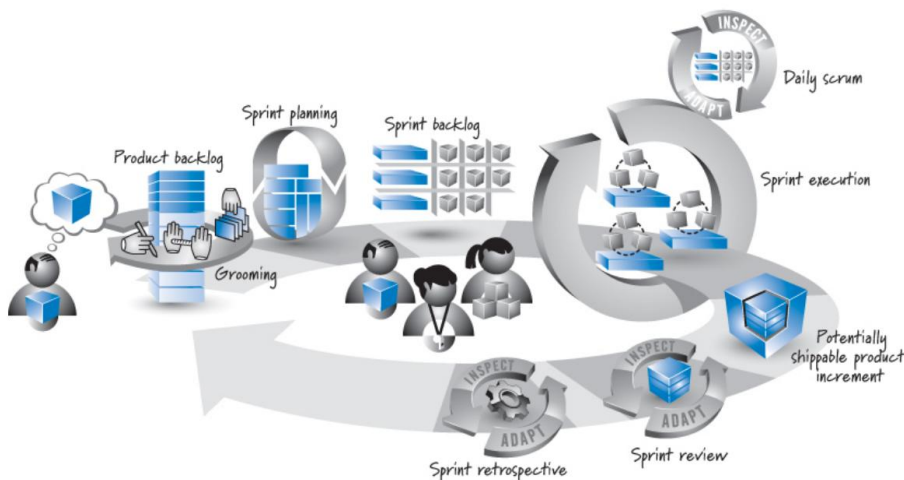


Figure 19: Different artifacts and ceremonies

The first step was establishing time-boxed sprints, each with clear goals and defined deliverables. Our sprints normally ran from Monday to Friday, except for the final sprint, where we shortened the duration due to time constraints. Ruben (2012) mention they always have a fixed start and end date, and generally they should all be of the same duration. We decided not to extend the sprint into the weekend so that all members could rest and recharge before continuing with the remaining work.

At the start of each sprint, we held a Sprint Planning meeting. During this meeting, we created and refined backlog items, discussed priorities, and estimated effort and importance. We used Planning Poker (Figure 20) to ensure that estimates were unbiased and independent. When significant differences appeared in individual estimates, we discussed the reasoning behind them before agreeing on a final point value.

- The chatbot flow works in the solution.
- The user can complete all the intended steps
- The text is clear and free of major errors
- It has been reviewed by at least two teammates
- It meets the acceptance criteria for a certain backlog item
- It is correctly updated in Scrumwise

Atlassian (n.d.) describes acceptance criteria as the conditions that a product, user story, or increment of work must satisfy to be complete. They're a set of clear, concise, and testable statements that focus on providing positive customer results. An overview of the acceptance criteria for all backlog items is provided in Appendix E.

Use of GitHub and Git Rules

Throughout the development of our solution, we used GitHub for collaboration and version control. Although the tool felt overwhelming at first, it quickly became an essential and efficient part of our workflow once everyone became familiar with it. Our repository was structured into two main directories — *frontend and backend* — which provided a clean separation of responsibilities. This structure made it easier for team members to focus on their respective areas and simplified debugging, testing, and general problem-solving.

To further avoid conflicts, we kept the frontend and backend on separate branches during development. Instead of merging continuously, we chose to perform a single major merge when both parts reached a stable stage. This approach accelerated the early development process and reduced the risk of merge conflicts while the structure was still rapidly evolving. Although this method would not be optimal for a long-term or large-scale project, it worked well within our short three-week timeline and allowed us to move quickly without unnecessary interruptions.

We set multiple Git rules to be followed when making changes to the project. The rules were as follows:

Rule 1: Never push directly to main. (This is enforced on GitHub, but do not attempt)

1.1 All new code must be integrated via Pull Request.

Rule 2: Use clear, descriptive branch names.

2.1 Branch names should reflect work done on that branch:

frontend-<short-description>: e.g. frontend-manual setup-alignment-fix

Rule 3: Branch from the main or latest relevant feature branch.

3.1 Always create a new branch from the latest, clean state of main (or relevant branch, i.e. frontend/backend)

3.2 Example workflow: Create a descriptive branch from the existing frontend branch -> frontend-style-adjustments -> commit your changes -> create a pull request to merge frontend-style-adjustments with frontend.

Rule 4: Write meaningful commit messages.

4.1 Commit descriptions should reflect changes made in that commit.

- Good: “Fixed manual input text field alignment”, “fix: Resolved infinite loop in API client”
- Bad: “fixed”, “improved descriptions”

Rule 5: Commit logical, complete code.

5.1 Each commit should introduce one complete functional change.

5.2 Avoid mixing multiple unrelated changes into a single commit.

Rule 6: Do not commit unfinished, untested or commented-out code.

6.1 Every commit should leave the project in a functional state. If this is not possible (due to e.g. time restraints), commit your current work to its related branch, but do not create a pull request.

Rule 7: When in doubt, branch out.

7.1 If you are unsure what changes have been made or what state the project is in, create a new branch.

Refrain from merging your own changes if you have the ability to. Always get someone else to approve your pull request.

Following these rules helped keep our working repository clean and prevented situations where one person could merge a change that would break the application, since each pull request required review from another collaborator.

Each day, we conducted a daily scrum (daily stand-up) followed by a short Q&A session. The stand-up meetings were brief and focused: each member shared what they had completed the previous day, what they planned to work on next, and whether they faced any obstacles. This meeting also allowed us to share relevant information, align expectations, and maintain a shared understanding of our progress. The daily scrum is essential for helping the development team manage the fast, flexible flow of work within a sprint (Ruben, 2012). By briefly reviewing the burndown chart each day (Figure 22), we ensured that everyone stayed aware of our pace and could react to any deviations early.

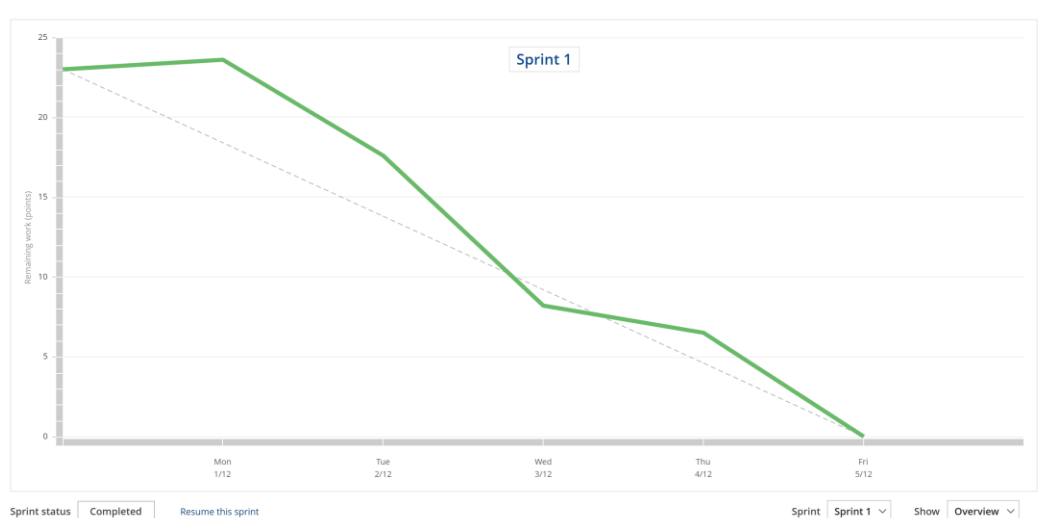


Figure 22: Burndown Chart in Sprint 1

After the stand-up, the Scrum Master joined the Q&A session arranged by the lecturer in which questions collected from the group were addressed. This allowed us to clarify uncertainties related to the workflow, the project, or general Scrum practices.

At the end of each sprint, we conducted both a Sprint Review and a Sprint Retrospective. The purpose of the Sprint Review is to inspect and adapt the product that is being built (Ruben, 2012). While Scrum typically involves stakeholders or the Product Owner testing the solution, we instead used a dedicated user to assist with this process. Their feedback helped us refine the solution and generate new backlog items for the next sprint.

The Sprint Retrospective focused on reflecting on how we worked together and identifying improvement areas (Ruben, 2012). Each group member had the opportunity to share their perspective, ensuring that any challenges, misalignments, or suggestions could be discussed openly. This practice helped us maintain cohesion within the team and continuously improve our collaboration and workflow.

How has the use changed throughout the project?

From the first sprint to the last, our use of Scrum evolved as we gained experience with the framework and our tools.

One of the biggest improvements was our use of Scrumwise. During the first sprint, several team members forgot to log hours or to assign tasks to themselves, which caused the burndown chart to appear inaccurate. As the project progressed, the team became more disciplined in updating tasks and registering time, resulting in a burndown chart that reflected our real progress much more reliably.

Another change was the way we handled daily stand-ups. Although the planned meeting time was 10:00, we often needed to adjust the timing when members were delayed. In those cases, the Scrum Master provided a short recap to late arrivals to keep everyone aligned. Team members who were absent due to work or sick leave received updates via Discord, ensuring that the entire group remained synchronised.

Another change throughout the project was how we balanced the number of items added to the sprint backlog. In the beginning, we were unsure whether we were selecting too many or too few tasks, so we started cautiously. This defensive approach led to periods where some team members had little to work on. As we gained more confidence in our capacity and workflow, we became more proactive and committed to a larger set of tasks each sprint. This shift also allowed us to include more testing-related items in the backlog, which provided valuable input on our solution and helped generate additional tasks for future sprints.

Overall, our use of Scrum became more structured, consistent, and better adapted to the realities of our team as the project moved forward.

What choices have you made?

React, Node.js and Vite

We chose React as our frontend framework primarily because it remains the most widely used option in the field. Although its growth has slowed in recent years, it still holds a dominant position among frontend frameworks (The Software House, 2024). Since our group has limited frontend development experience, selecting the most established and well-documented framework made the learning process more manageable.

Using Node.js became a natural consequence of choosing React. Modern React development depends on a Node-based toolchain for package management, building, and running the development server. In our case, Node.js is not used at runtime in the final application but strictly as supporting infrastructure during development. We did not consider alternative JavaScript runtimes, as Node.js is the standard and most widely supported environment for React projects (Chandra, 2025).

For the build system and development server, we selected Vite. It has become a popular choice for modern React applications and has largely replaced older tools such as Create React App. We chose Vite for its fast startup times, efficient hot-reloading, and minimal configuration needs (Comachio, 2024). More opinionated frameworks like Next.js were intentionally avoided since

our project does not require server-side rendering or built-in backend capabilities, and keeping the frontend simple aligned better with our project scope.

Database:

We chose SQLite as our database because it is lightweight, serverless, and works cross-platform, allowing every team member to run the project locally on both macOS and Windows without needing to install or configure a separate database server (Hipp, 2025). This made development significantly simpler and reduced setup time for the group. Another advantage is that SQLite stores everything in a single file, which makes it easy to share, reset, or back up during development (Hipp, 2025).

We paired SQLite with SQLAlchemy because it keeps the data layer clean and allows us to interact with the database using Python classes instead of raw SQL (SQLAlchemy, n.d.). This improves code readability, reduces errors, and gives us the flexibility to switch to a more scalable database (such as PostgreSQL) in the future with minimal changes (SQLAlchemy, 2024). One limitation of this setup is that we did not use migrations; therefore, if the database schema changes, the local database must sometimes be recreated to match the updated model.

All project data is stored in an SQLite file called *events.db*, which is accessed through SQLAlchemy ORM. SQLAlchemy acts as an Object–Relational Mapper, allowing us to treat database rows as Python objects. The backend uses a SessionLocal session factory: whenever an API endpoint needs to read or modify data, it opens a session, calls the appropriate CRUD function from *crud.py*, commits any changes, and then closes the session.

The database schema is defined in *db/schema*. The central table is *Event*, which acts as the root entity, with related tables such as *Participant*, *EventImage*, and *ConversationModel* linked through foreign keys. On application startup, *Base.metadata.create_all()* ensures that all required tables exist.

Python (Backend):

The backend is implemented in Python using the FastAPI framework. In practice, the frontend sends HTTP requests such as “create event,” “update sidebar field,” or “upload participants,” and FastAPI exposes these as endpoints (GeeksforGeeks, 2025). Each request is validated against Pydantic models, which define the expected data structure and ensure that the input is correct before processing (GeeksforGeeks, 2025). After validation, the endpoint calls our internal handler or CRUD functions to interact with the database. We chose Python because it is easy to read, quick to develop with, and well-suited for our project with many interconnected components such as API endpoints, database logic, file handling, and AI integration. Python also has a strong ecosystem for web development, data processing, different libraries, and AI tooling, allowing us to build the entire backend in one consistent and maintainable language (Zoon, 2025).

Gemini (AI assistant):

We use Google Gemini as the AI assistant that guides users through the event setup process via chat. The idea is simple: the user writes in natural language (for example, “Create a soccer tournament in Oslo next week”), Gemini interprets the message, and returns structured tool actions such as “set event name,” “set date,” or “set location” (Gemini API (a), 2025). Our backend then executes these actions by updating the in-memory event state and storing the information in the database once sufficient details have been collected.

We chose Gemini because it’s well documented, easy to understand and allowed us to integrate the model efficiently (Gemini API (b), 2025). In addition, Gemini supports structured, tool-based workflows, meaning the AI can update real fields rather than only generating text responses (Schmid, 2025). This made it a strong fit for an interactive assistant that modifies actual event data. Gemini also provides good output quality at a reasonable cost for this type of application (Gemini API (c), 2025).

One limitation we experienced is that the API occasionally returns slow or incomplete responses when the servers are under heavy load. This can be mitigated by implementing retry and backoff strategies, which help stabilise the interaction flow during temporary service disruptions (Gemini Apps Help, 2019).

Theoretical choices

In our theoretical choices, we focused on applying principles from both Scrum and user-centred design. The use of sprints, daily standups, and sprint reviews was grounded in agile theory, emphasising transparency, iterative development, and continuous improvement. On the design side, we aimed to minimise cognitive load by guiding users through a step-by-step onboarding process supported by clear navigation and visual hierarchy. We also applied conversational UI theory by introducing an AI assistant, allowing users to interact with the system using natural language rather than complex forms. The database design followed relational theory, using a normalized structure with an Event as the root entity and related tables linked through foreign keys to maintain consistency and scalability. These theoretical choices guided the structure, design, and workflow of the entire solution.

Design

Since our solution was developed from the ground up, we decided to build a deliberately simple UI (User Interface), prioritising functionality over visual refinement in the early stages. The design is built around rounded, square-shaped components, with clear visual separation between different areas of functionality. This structure was chosen to minimise confusion and make the interface as intuitive as possible.

We decided on a general dark theme with green accents, inspired by Peers' current landing page, to somewhat keep a visual consistency with their brand.

A floating navigation bar is present at the top of all pages. The intent is to always keep navigation accessible to the user, allowing them to move between sections at any point without needing to scroll back to the top. The navigation bar follows the user as they scroll, ensuring constant visibility.

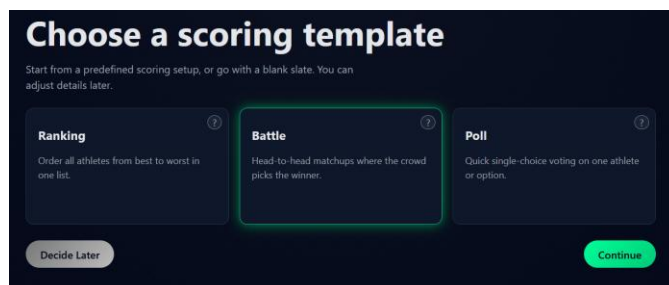


Figure 23: Scoring template page, featuring a multi-column layout.

Apart from the home page and the dashboard, all pages share a similar layout. Each page begins with a prominent title indicating the page's purpose, followed by a short description, further explaining the page. Below this, the content is organized into grid-based layouts, tables, or multi-column sections, depending on the page's function. An example of a multi-column page layout can be seen above in Figure 23.

Have you deviated from recommendations? Why?

Throughout the project, we made a few deviations from standard Scrum recommendations to adapt the framework to our workflow, team size, and time constraints.

First, we only involved a single user during our Sprint Reviews. Ideally, Scrum encourages participation from multiple stakeholders or a Product Owner, but given the scope of our project, we chose one consistent user who could provide continuous and reliable feedback. This allowed us to maintain testing efficiency while still receiving meaningful input.

We also did not strictly adhere to time-boxing for all ceremonies. Some meetings extended longer than intended, especially early in the process when team members needed extra clarification. These extensions helped the group establish a common understanding and were beneficial during the initial stages of the project.

Another deviation occurred when we added a few backlog items during ongoing sprints. While Scrum recommends freezing the sprint backlog after commitment, there were moments we felt it was necessary to add some items to the current sprint. If possible, we would write up the backlog item and add it to a later sprint. Those we initiated in the current sprint were for us to maintain momentum without the possibility of delaying future sprints by adding them later.

Lastly, many of our backlog items were technically oriented rather than written as user stories. In a typical Scrum environment, backlog items are usually framed around user value, but given the academic context and the technical nature of our assignment, working with technical tasks was more practical and provided clearer guidance for the team. Many of these tasks were

directly derived from the sprint questions we defined during the sprint design, as well as the overall project context provided to us at the start. This approach ensured that our work aligned with the learning outcomes and technical requirements of the course.

Overall, these deviations were intentional adaptations made to support effective teamwork, maintain productivity, and accommodate the constraints of a student project.

Group Reflection on the Scrum Process

As a group, we experienced the Scrum process as both valuable and challenging throughout the project. While the framework provided structure, clarity, and a shared direction, our varying levels of experience strongly influenced how effectively we were able to use it, especially in the beginning. With most of us being inexperienced with the Scrum Process itself but also the lack of diversity since we all are studying the same field.

One of the main reflections shared across the group was that the Scrum Master role carries a significant workload. Ensuring that everyone stayed aligned, understood the process, and used the tool correctly required more coordination than expected. Early uncertainty and lack of experience also contributed to a somewhat bumpy start. Communication within the group was generally good, but we also saw how easily misunderstandings could arise when responsibilities or expectations were unclear.

For several group members, the Scrum process highlighted what it means to work dependently within a team. When certain tasks relied on others being completed first, waiting naturally occurred, which at times created frustration or slowed individual progress. This dependency, however, also reinforced the importance of communication, prioritisation, and transparent division of work.

Scrumwise itself was generally perceived as intuitive and helpful. Once we became familiar with it, the tool provided a clear overview of progress, responsibilities, and priority areas, making it easier to stay organised. That said, some felt that we did not fully utilise all the features available, especially early in the project when many were still learning the tool. Initially, it was

also challenging not to add new backlog items directly into an active sprint, which occasionally led to feelings of inefficiency and frustration.

Across the group, there was agreement that engagement fluctuated at times. Some felt they started strong but became less active later, while others needed time to build confidence before contributing more effectively. Despite this, everyone recognised the value of the Scrum process and how our understanding of it improved over time. As a group, we learned the importance of estimating realistically, keeping tasks small, and updating the task board regularly.

Overall, the experience highlighted both the potential and the learning curve of Scrum. While the process confronted us with challenges — including coordination, prioritisation, and adapting to a new workflow — it also gave us structure, visibility, and a clearer sense of collaboration. It is a methodology several of us can see ourselves using again in future projects, now with greater insight into how to apply it more effectively.

4. The Way Forward

Despite the strong foundation achieved during the project, the group agrees that the solution isn't fully finished and there is much potential for improvement. The primary limitation identified was time, which prevented the completion of several smaller details and remaining features. Some functionality is only partially implemented, such as participant handling, where data is stored in the database but not fully visualized in the user interface, as well as certain dashboard features that are currently missing or incomplete. Additionally, since the solution was developed as a web application, it is not fully optimized for mobile use.

Additionally, a feature for sharing events across different platforms was designed but not fully implemented before the end of the project. With more development time, this functionality could be completed and integrated as part of the event management flow. We would further investigate the participant registration process and the sponsor's part of the registration.

The AI assistant, while functional and central to the solution, would benefit from further refinement and expansion. At times, responses can be slow, and support for more advanced

workflows—such as deeper participant and event linking—is limited. Continued development would include improved training, retry and fallback mechanisms. Related features such as file uploads and automatic data extraction, dynamic previews, and sport-specific templates are currently conceptual and would require further implementation to reach production readiness. From a usability perspective, the frontend design was intentionally kept minimal; however, feedback indicates that it may feel too barebones for some users. Users unfamiliar with the event setup process may lack sufficient guidance, potentially leading to confusion. Future work should therefore place greater emphasis on usability improvements, clearer onboarding, and visual refinement.

Figure 24 below shows backlog items that we would have addressed in future sprints and that, in our view, would have further improved the solution.

Frontend: Technical Implementation Epic ^	Backend: Technical Implementation Epic ^
Remove most inline css and create more reusabl... 3 points	AI change summary payload 2 points
Fix card coloring in SetupTemplate, so it matches... 1 point	AI “fallback mode” when busy New
Improve styling in SetupManual, to make it appe... 2 points	Roles & permissions New
Change the participants table into a scrollable... 2 points	Audit log for changes New
Choice between day and night mode 2 points	Centralized error logging 3 points
Make the 'About' and 'Contact' buttons link to ou... 2 points	Version database schema changes safely over... 2 points
Warn if important info is missing or inconsistent. New	Backup/restore routine 3 points
Create a new event by copying an existing one. 1 point	Make the AI - DB communication faster 3 points
Export event + participants (PDF/CSV) 2 points	Tune the systemprompt even more 2 points
Share an event view without allowing edits. 1 point	Connect a spesific event to a specific user 3 points
Switch UI language. 3 points	When connected to a spesific user, In dashboard display... New
Make compatible with different devices 3 points	More abuse protection 2 points

Figure 24: Future backlogs items

Part 3 Reflection Note

Group Reflection Note

Communication within the group worked very well throughout the project. Meeting physically almost every day helped us coordinate quickly, resolve misunderstandings, and maintain a shared understanding of priorities. The Discord server also played an important role in keeping everyone updated, especially when someone arrived late or was unable to attend. Overall, we managed to create an open and honest environment where concerns could be raised freely.

The division of work was generally fair, and most of us were able to contribute to areas that matched our strengths. Splitting the team into a frontend and backend group gave us focus and helped maintain progress. However, this structure also introduced challenges: when one team finished early or got blocked, it was not always easy to contribute to the other side due to the complexity of tasks. At times, dependencies between tasks meant some members had to wait for others to complete key functionality. With the size of the group, it felt at times that the scope itself was a bit too small for a group with seven people.

Regarding professional competence, the solution reflects our current skill level well. None of us had previously built a project of this scale, and many technologies were new — including FastAPI, React, SQLAlchemy, AI-assisted workflows, and integration processes. The background of our group is Data Science and shows both our strengths and weaknesses in what we already know and have to learn to make the solution work. Considering this, we are satisfied with what we delivered. While the final product is not fully polished and some features remain incomplete, the solution demonstrates that we can learn quickly, apply new tools, and collaborate effectively to build a functioning system under time pressure.

For future projects, several improvements were identified. First, we would prioritise merging work more frequently to avoid large and complex integration steps. Second, we would invest more time in upfront planning, particularly around design, user flow, and task dependencies. Having clearer sketches, architectural diagrams, and shared expectations early on would have

prevented many of the bottlenecks we experienced. Third, we would introduce more frequent user testing rather than relying solely on sprint reviews, as even brief mid-sprint feedback sessions can significantly improve the project's direction. Finally, we would conduct more research in advance. Although this project provided a valuable learning experience, we often found that a deeper initial understanding could have helped us work more efficiently and deliver stronger solutions in certain areas.

The three most important lessons we learned include:

1. **The value of structured processes** — tools like Scrumwise with clear backlogs and burndown charts, and daily standups reduce confusion and help coordinate a large team.
2. **The importance of breaking tasks down** — smaller, well-defined items made progress more manageable and collaboration easier.
3. **The impact of user testing** — user feedback often shifted our priorities and revealed misunderstandings we were too close to the project to see.

Overall, the project was an intense and steep learning process, but a rewarding learning experience. Despite challenges, we grew both technically and collaboratively, and we leave the project with a much stronger understanding of teamwork, planning, and real-world software development.

Individual Reflection Notes

Each member will be submitted in a separate note on Canvas.

Reference list:

Atlassian. (n.d.). *Acceptance criteria: examples and best practices*. Collected 15. December 2025 from:

<https://www.atlassian.com/work-management/project-management/acceptance-criteria>

Chandra, Y. (2025, March 24.). *JavaScript usage statistics: How the web's favorite language fares in 2025*. ZenRows.

<https://www.zenrows.com/blog/javascript-usage-statistics-most-used-language>

Comachio, E. (2024, December 19.). *The developer experience upgrade: From Create React App to Vite*. Tweag.

<https://www.tweag.io/blog/2024-12-19-cra-to-vite/-:text=toolchain,of JS 2024 Developer Survey>

GeeksforGeeks. (2025, July 23.). *FastAPI - Pydantic*.

<https://www.geeksforgeeks.org/python/fastapi-pydantic-2/>

Gemini Apps Help. (2019, October 9.). *gemini 2.5 pro returning 503 error*.

<https://support.google.com/gemini/thread/379296646/gemini-2-5-pro-returning-503-error>

Gemini API (a). (2025, December 18.). *Function calling with the Gemini API*

<https://ai.google.dev/gemini-api/docs/function-calling?example=meeting>

Gemini API (b). (2025, December 18.). *Gemini API quickstart*. <https://ai.google.dev/gemini-api/docs/quickstart>

Gemini API (c). (2025, December 18.). *Gemini models*.

<https://ai.google.dev/gemini-api/docs/models#gemini-3-flash>

Hipp, D. R. (2025). *About SQLite*. SQLite.

<https://sqlite.org/about.html>

Rubin, K. S. (2012). *Essential Scrum: A practical guide to the most popular Agile process*. Addison-Wesley.

Schmid, P. (2025, March 5). *Function Calling Guide: Google DeepMind Gemini 2.0 Flash*.

Philschmid.de; Philipp Schmid. <https://www.philschmid.de/gemini-function-calling>

SQLAlchemy. (2024, 5. December). *Object relational tutorial (1.x API)*.

<https://docs.sqlalchemy.org/en/14/orm/tutorial.html>

SQLAlchemy. (n.d.). *Features and philosophy*. Collected 18. December 2025 from:

<https://www.sqlalchemy.org/features.html>

The Software House. (2024). *State of Frontend 2024*.

<https://tsh.io/state-of-frontend>

Zoona. (2025, August 4.). *Python Development in 2025: Why It's the Go-To Choice for Scalable Web Applications*. Codism.

<https://codism.io/python-development-in-2025-why-its-the-go-to-choice-for-scalable-web-applications/>